TRANSFORMED 2 SUCCEED

transformed2succeed.com

FREE LIVE SESSION · 30 MINUTES

GPU Optimization for AI-ML Systems

Make AI models faster and cheaper to run, and prove it with numbers. No GPU required.

PRESENTER Dr. Emmanuel Naweji **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of GPU Optimization for AI-ML Systems, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

THE ONE IDEA

Measure first. Change one thing. Prove it with numbers.

01

Can't cut what you can't see

Cost per 1M tokens, the real bottleneck, and what the GPU is actually doing.

02

Precision is a dial

Shrink the model until quality moves, then stop. Always with a quality number.

03

Serving is a memory problem

Same model, same GPU, more users: stop wasting memory and keep batches full.

COURSE MAP

Three modules, one habit: measure first

MODULE 1

Measure Before You Optimize

Cost per 1M tokens, compute vs memory-bound, and a before-and-after you can prove.

MODULE 2

Shrink the Model

Quantize, distill, prune and route, with quality measured on your own eval set.

MODULE 3

Serve It Smarter

KV cache math, paged attention and continuous batching: more users per GPU.

01

MODULE 1 OF 3

Measure Before You Optimize

Every optimization starts with a number. This module gives you the numbers.

Count the cost of one unit, not the fleet

$0.95

per 1M tokens: $288 of A100 time ÷ 302M tokens

17%

of doc-summarizer spend bought zero requests

−25%

cost per 1M tokens after one batching change

Cost per 1M tokens = cost per hour ÷ tokens per hour × 1,000,000. Count idle hours too: you pay for them.

Two phases, two different bottlenecks

PREFILL

Compute-bound: reading the prompt

All prompt tokens at once: lots of math per byte loaded. You feel it as time to first token.

VS

DECODE

Memory-bound: writing the answer

One token at a time, and every token re-reads every weight. Bandwidth sets the pace.

GPU marketing sells FLOPs. For chat workloads, memory bandwidth usually limits you. Check it before you buy.

Compute-bound or memory-bound?

TERMINAL

```
$ python3 bottleneck.py --params 8e9 --precision fp16
--gpu L4
$ python3 bottleneck.py --params 8e9 --precision int8
--gpu L4
```

`--params 8e9`
an 8-billion-parameter model

`--precision`
bytes per weight: fp16 = 2, int8 = 1

`--gpu L4`
24 GB, 300 GB/s: from the GPU catalog (--list)

`the ceiling`
a speed limit, not a prediction

YOU SHOULD SEE fp16: 16.0 GB, memory-bound, ~19 tokens/s per user · int8: 8.0 GB, ~38 tokens/s per user

“100% utilization” can mean very little

01

GPU 0: 100%, 142/400 W

Busy but not productive: a kernel ran, mostly stalled on memory.

02

GPU 2: 0%, 61/400 W

Idle, and you are still paying for this GPU.

03

GPU 3: 99%, 399/400 W

Throttled: power-capped and hot (87C). Quietly slower than the rest.

Check SM and tensor-core activity, power vs. limit, and tokens/s per GPU. Busy is not productive.

Your lab: a performance scorecard

1

Baseline scorecard

Unit costs for two services, and the spend that bought nothing.

2

Find the bottleneck

bottleneck.py: the roofline ceiling, and what sets it.

3

Read the GPU

gpu_snapshot.py --sample: utilization, power, throttling.

4

Change one thing

--compare after data: −25% cost, −20% p95 TTFT. Write FINDINGS.md.



Same day. Same traffic. One change. 25% cheaper, with proof.

Two changes at once means you can't explain either one.

02

MODULE 2 OF 3

Shrink the Model

Precision is a dial, not a switch. Turn it until quality moves, then stop.

Settings on the precision dial (8B model)

32 GB

FP32 · 4 bytes: training master copy, rarely served

16 GB

BF16 / FP16 · 2 bytes: the usual LLM default

8 GB

FP8 / INT8 · 1 byte: FP8 math needs Hopper/Ada

≈ 4 GB

INT4 · 0.5 byte: biggest win, most risk

Module 1: half the bytes, double the memory-bound ceiling. 8B on an L4: ~19 → ~38 tok/s.

Who shares a scale decides everything

INT8 PER-TENSOR

−18.5 pts (rare slice −34.8)

One scale for the whole matrix. Six outlier channels stretch it; normal weights round to almost nothing.

VS

INT8 PER-CHANNEL

−0.1 pt

One scale per neuron, so an outlier only hurts itself. INT4 group-128: −0.3 pt, 7.7x smaller.

Same bits, same 2.18 MB. Measure on your own eval set and a rare slice, not a leaderboard.

Turn the precision dial

TERMINAL

```
$ python3 data/make_data.py
$ python3 quantize.py
```

make_data.py
build the task, train a 2.2M-param teacher (~20–60 s)

quantize.py
store the teacher in every format and measure it

acc / rare
accuracy overall and on the rare slice

change, pts
points lost or gained vs FP32

YOU SHOULD SEE INT8 per-tensor −18.5 / −34.8 <- moved · INT8 per-channel −0.1 / −0.1 · INT4 group-128 −0.3 / −0.3

Three more levers, each with a quality number

01

Distill

A student 113x smaller keeps 98% of the teacher's accuracy. Rare slice: −2.8 pts.

02

Prune structured, then heal

Half the neurons: 48.9% → 90.7% after 2 epochs, 3.1x faster.

03

Route

Small model first, escalate when unsure. At 0.70, 89% stay on the student.

Unstructured 50% zeros ran at 0.9x speed: a dense matmul still multiplies every zero.

Your lab: one change at a time

−94%

modelled cost: $0.95 → $0.06 per 1M tokens

−0.1 pt

accuracy, on the held-out test set

−0.7 pt

rare slice: you know because you measured it

Step 1: teacher INT8 per-channel halves the cost. Step 2: route 89% to the INT8 student.



Never report a compression win without a quality number.

Turn the dial until quality moves, then stop.

03

MODULE 3 OF 3

Serve It Smarter

Same model, same GPU, more users: stop wasting memory and keep every batch full.

Weights decide if the model fits. KV decides users.

128 KiB

KV cache per token: Llama-3-8B, FP16

1 GiB

per user at 8K context

3

full-length users on a 24 GB L4

50

full-length users on an A100-80GB

Bytes per token = 2 × layers × KV heads × head_dim × bytes = 2 × 32 × 8 × 128 × 2 B.

How many users fit on one GPU?

TERMINAL

```
$ python3 kv_calc.py --model llama-3-8b --context
8192 --users 32

$ python3 kv_calc.py --compare --gpu A100-80GB --
context 4096
```

`--model`
model shape from public configs

`--context --users`
tokens held per user, and how many users

`budget`
90% of memory, minus weights, minus 2 GB overhead

`--compare`
no GQA vs GQA vs FP8 KV on one GPU

YOU SHOULD SEE 32 users = 32.0 GiB of KV + 15.0 GiB of weights · L4: 3 seqs at 8K · A100-80GB: 50 seqs · GQA 3.8x, FP8 2.0x

Stop waiting for the slowest answer

STATIC BATCHING

p95 TTFT 31.5 s

Take a group, run it until the longest answer ends. Short answers leave idle slots; new requests wait.

VS

CONTINUOUS BATCHING

p95 TTFT 0.10 s

The batch is re-decided every decode step. Finished requests leave at once; waiting ones join.

Same simulated GPU, same traffic, max batch 256: 1,495 → 4,599 tokens/s at capacity (+208%).

Less waste, less repeated work

01

Paged attention

16-token blocks, like OS pages. Waste 66.5% → 0.8%; 134 → 255 running.

02

Prefix caching

Stable prompt first: 92% hits. One timestamp at the top: 0%.

03

Speculative decoding

Cheap draft guesses, big model checks: 2.8x at α 0.8. Hard text: 0.82x.

Serving is a memory problem before it is a math problem.

Your lab: size the cache, run the sims

1

Size the KV cache

kv_calc.py: bytes per token, per user, users per GPU.

2

Page the cache

paged_sim.py: max-length slabs vs 16-token blocks.

3

Batch it

batch_sim.py: static vs continuous, then -- sweep to find the knee.

4

Cache and speculate

prefix_cache_sim.py and spec_decode.py: when each pays, and when not.



Same GPU. Same model. Same traffic. 3× the capacity.

Before “buy more GPUs”, size the cache and fix the scheduler.

RECAP

Three modules, three habits

Measure before you optimize

Cost per 1M tokens, the real bottleneck, and one change, re-measured.

Shrink the model, not the quality

Per-channel quantization, distill, prune, route. Always a quality number.

Serve it smarter

Size the KV cache, page it, batch continuously: 3x capacity, same GPU.



You can make AI faster and cheaper, and prove it.

Every module ends with real numbers, a portfolio file and an interview story.

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- Zero to AI-ML Systems Engineering
- DevOps for AI-ML Systems
- SRE for AI-ML Systems
- **GPU Optimization for AI-ML Systems**
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	Linux for AI-ML Systems	Zero to AI-ML Systems Engineering	SRE for AI-ML Systems	NEXT FinOps for AI-ML Systems
THURSDAY	Python for AI-ML	DevOps for AI-ML Systems	TONIGHT GPU Optimization for AI-ML Systems	AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Monday, 7:30 PM Central · FinOps for AI-ML Systems

Questions? Let's take them now.

Dr. Emmanuel Naweji · Transformed 2 Succeed